



Christophe PICHAUD
Architecte Microsoft chez
'Modern Applications by Devoteam'
christophepichaud@hotmail.com
www.windowscpp.com



C++

C++ et Containers Docker sur Azure

Pour un Gold Partner ayant des bureaux à Paris, j'ai réalisé une vitrine technologique Azure mettant en œuvre les technologies Microsoft .NET, C#, C++, LMDB, Win32 et Docker, puis la déclinaison du run d'une instance Docker sous Azure.

L'histoire commence avec la récupération de sources C++ sous brevet, le code d'une base de données non terminée... Ce projet était intitulé la base de données la plus rapide du monde. Il était question de moteur de cube OLAP et de reporting BI pour concurrencer SQL Server SSAS et Tableau Software... Après investigation dans le source code pendant un mois, je constate que c'est un beau projet mais qu'il n'est pas terminé et que cela va être compliqué de réaliser la partie manquante car le code est assez complexe et surtout dans quelles conditions et pour quel résultat ? En accord avec mon patron, on décide de rechercher des composants réutilisables. Le moteur de stockage attire mon attention. Or il se trouve que c'est une librairie open-source nommée OpenLDAP-LMDB qui existe sous Linux ; c'est un filesystem en mémoire... LMDB a été couvert dans le numéro de Mars 2019 de Programmez. Cherchez le paquet LMDBNet sur nuget : <https://www.nuget.org/packages/LMDBNet>

Architecture Logicielle 1

Voici le soft qui va être construit ; c'est un service Windows qui embarque un serveur web autonome sur lequel on a greffé un Web Service REST API JSON, le tout construit en C++ avec le Microsoft

C++ REST SDK. Le Web service se publie sur le port 7001. Il faut être administrateur pour pouvoir créer le port. C'est une restriction WinNet.

Il s'agit d'un serveur web embarqué qui expose un web service REST API qui permet de gérer un ou plusieurs cache NoSQL LMDB. Le projet est un Windows Service x64 (LMDBService.exe) qui charge plusieurs dll :

- cprestd141d_2_10.dll
- LMDBWindowsDIID64.dll
- LMDBWrapperD64.dll
- MySharedStuffD64.dll
- Msvcp140d.dll
- Vcruntime140d.dll
- Ucrtdbased.dll

Vous aurez remarqué au passage que les modules sont en debug... C'est pour faciliter le debugging.

Run en local

Le service Windows se lance via net start ou en mode console... Il consomme 2 MB de mémoire et le CPU est toujours à 0 %. C'est l'avantage du C++.

Console – lancement du démon

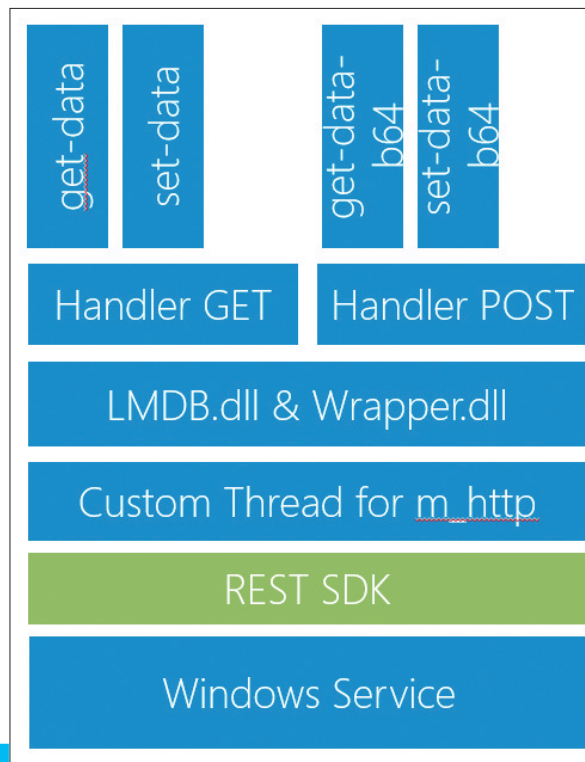
On teste le serveur web embarqué depuis Chrome.

Mise en œuvre de Docker

Avant tout, il faut installer Docker sous Windows. Ce n'est pas très compliqué. On lance le setup et c'est terminé. Le premier problème est de savoir comment créer une « application » pour Docker. Et là, c'est un moment de solitude car la documentation Docker est très mal faite, c'est une honte. Heureusement, je mets la main sur un ouvrage intitulé « Docker on Windows ». Une chance. Le livre explique que Docker est un système de virtualisation dans lequel une image vierge du système d'exploitation se voit greffer des composants via un fichier de boot (le dockerfile) et peut les exécuter comme une VM et dès que le container s'arrête, on perd tout. On peut recommencer à l'infini.... De manière isolée.

La base : le DockerFile

Un DockerFile est un fichier de boot pour l'image Docker. On va lui indiquer la base de l'operating system sur laquelle on boot et les différents paramétrages applicatifs à y ajouter. Avant de monter une instance de container Docker sous Azure, il faut d'abord maîtriser son fonctionnement en local avec un Docker en local, par exemple sous Windows 10. Voici mon dockerfile qui utilise une image Windows Sever 2016 LTSC (Long time support). Le DockerFile est le suivant :



```
FROM microsoft/iis:windowsservercore-ltsc2016
#RUN netsh advfirewall set allprofiles state off
COPY *.*/
RUN sc sdset SCMANAGER D:(A;;CCLCRPRC;;;AU)(A;;CCLCRPWPRC;;;SY)(A;;KA;;;BA)S:(AU;FA;KA;;;WD)(AU;OIIOFA;GA;;;WD)
RUN sc create LMDBService start=auto binpath="C:\LMBService.exe"
#EXPOSE 80
EXPOSE 7001
RUN md c:\temp
```

On part d'une image d'OS fournie par Microsoft qui contient IIS...
Les étapes sont les suivantes :

- Copie des binaires sur l'image ;
- Mise à jour des droits admins sur le SCManager ;
- Création du service ;
- Ouverture des ports 80 et 7001.

Run en local sous Docker

Ensuite, il faut builder l'image Docker :

```
docker image build --tag mydocker/myserver d:\dev\docker
```

Ensuite, on peut lancer le container en démon en ouvrant le port 7001 créé par le service :

```
docker run -d -p 7001:7001 -it mydocker/myserver
```

Pour pouvoir tester le container, il nous faut son adresse IP avec son ID en utilisant :

```
docker ps -a
```

On prend le premier item de la liste. On lance la ligne suivante avec son id à la fin :

```
docker inspect -f "{{.NetworkSettings.Networks.nat.IPAddress }}" 543e57f54047
```

La réponse s'affiche => 172.26.255.203

Il est maintenant possible de tester le container. **2**

Pour arrêter le container, il faut faire :

```
docker stop 543e57f54047
```

Comme vous pouvez le constater, Docker c'est quasiment indolore et transparent. Docker permet de faire fonctionner tout ce qui est non graphique. Les services de communications, les daemons, les

services Windows sont des candidats idéaux pour fonctionner sous Docker. Exemple : faire tourner un SQL Server de développement. C'est un service Windows qui utilise le port 1433. IIS qui est un site web qui utilise le port 80.

Run dans Azure

Là, les choses se compliquent un peu... D'abord, pour utiliser Azure, il faut une souscription. Ensuite il faut créer un registry. En effet, un container ne peut être instancié que s'il existe un registry ou est déposée l'image Docker.

Première étape, il faut créer un repository dans Azure Container Registry. Une fois créée, cette ressource donne un user/pwd. Sélectionnez l'item Access Keys et récupérez le mot de passe dans le clipboard : Connectez-vous à Azure Container Registry via son user/pwd depuis la console docker:

```
docker login lmbdreg2.azurecr.io -u lmbdreg2 -p V5nUpEG7Hu/V4KZ8kGy6hBT4r70tleF1
```

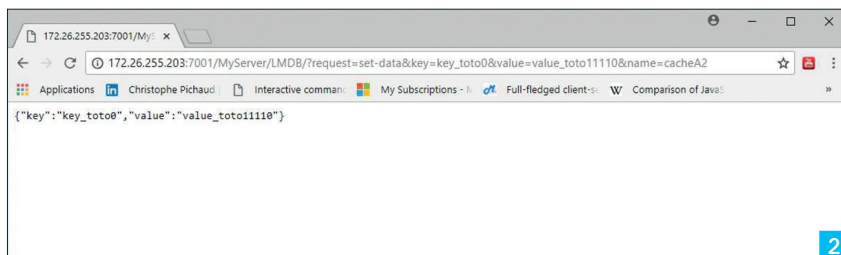
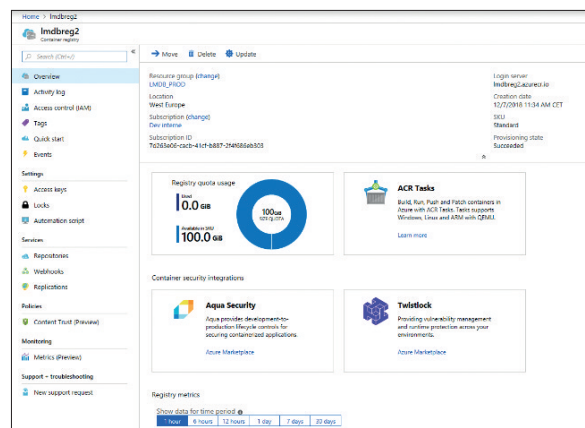
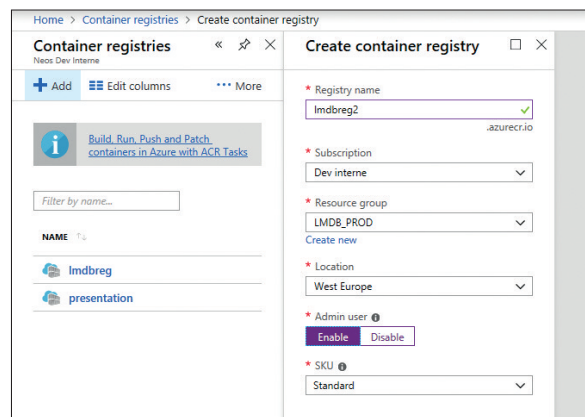
Ensuite, il faut tagguer le registry pour l'image Docker :

```
docker tag myserver lmbdreg2.azurecr.io/lmbd
```

Ensuite, il est possible d'uploader l'image docker vers ACR.

```
docker push lmbdreg2.azurecr.io/lmbd
```

La dernière commande envoie l'image dans ACR.



Créer une instance du container

Sélectionnez Container Instances et positionnez les informations de registry et les credentials. Cliquez sur OK. Sélectionnez Open additional port et tapez 7001. Cliquez sur OK.

Cliquez sur OK. Le container est déployé sur Azure ACI en 3 ou 4 minutes. Une fois déployé, on va sur le portail pour y récupérer son adresse IP en haut à droite. **3 4 5 6 7 8**

Mon service Windows contient un ping interne qui retourne son état. Faisons un test de ce ping logiciel :

<http://137.117.141.0:7001/MyServer/LMDB/?request=ping> **9**

Faisons aussi un About logiciel :

<http://137.117.141.0:7001/MyServer/LMDB/?request=about> **10**

Dans une version évoluée du service Windows, je crée un fichier de logs dans c:\temp\logs. Je définis mon dockerfile avec une image qui contient IIS. Puis je crée un vdir sur c:\temp\logs.

Ainsi, avec le port 80 depuis mon browser, je peux lire mes logs :

<http://137.117.141.0/Logs/lmdb.txt>

Problème réseau

Lors de mes premiers tests sur Azure, j'ai rencontré des difficultés à établir la connexion entre le browser et mon service Web. Rien ne marchait. J'ai trouvé une information sur le Web qui disait d'énu-

mérer les NICs (les cartes réseaux). Et en effet, il y a plusieurs possibilités. Soit créer les services sur l'ensemble des cartes réseaux, soit sur... la dernière !

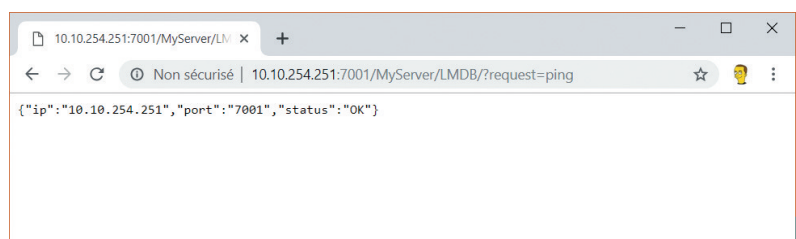
Voici le code qui itère sur les cartes et garde en mémoire la dernière adresse IP :

```
std::wstring ServerHelper::GetIP()
```

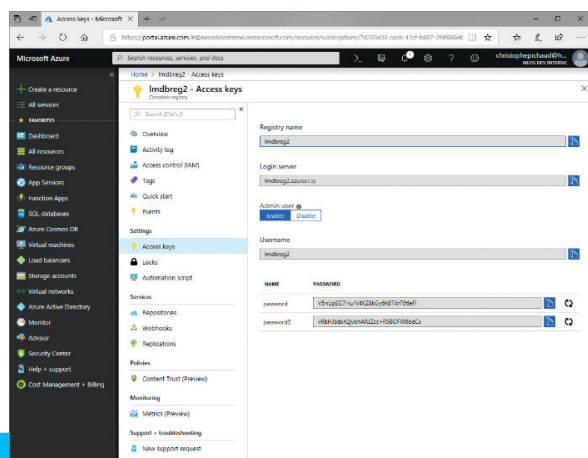
```
{
    //return L"127.0.0.1";
}
```



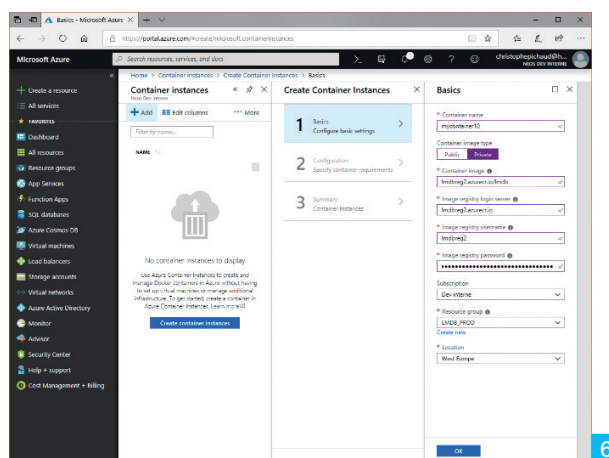
10



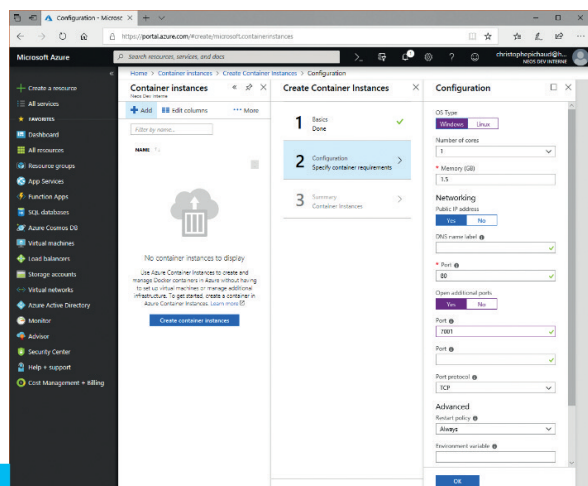
9



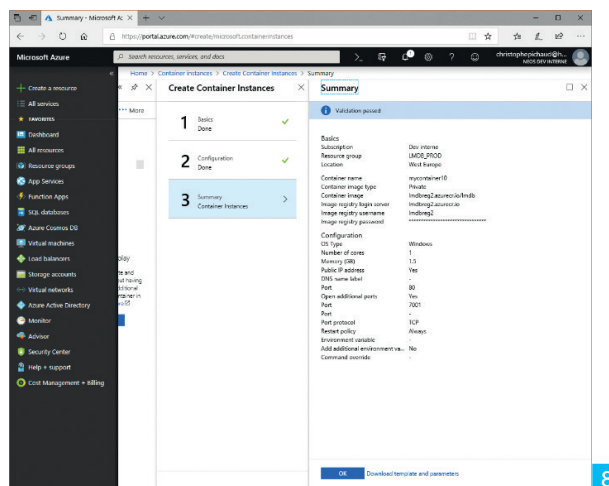
5



6



7



8

```
//return L"0.0.0.0";

// Init WinSock
WSADATA wsa_Data;
int wsa_ReturnCode = WSASStartup(MAKEWORD(2, 2), &wsa_Data);

// Get the local hostname
char szHostName[255];
gethostname(szHostName, 255);

struct hostent *remoteHost;
struct in_addr addr;
std::string ip;

remoteHost = gethostbyname(szHostName);

if (remoteHost->h_addrtype == AF_INET)
{
    int i = 0;
    while (remoteHost->h_addr_list[i] != 0)
    {
        addr.s_addr = *(u_long *)remoteHost->h_addr_list[i++];
        printf("IPv4 Address #%d: %s\n", i, inet_ntoa(addr));
        ip = inet_ntoa(addr);
    }
}

WSACleanup();
std::wstring ipw(ip.begin(), ip.end());
return ipw;
}
```

Cette routine est utilisée dans la procédure d'initialisation du service qui crée le serveur web et le service REST sur le port 7001 :

```
DWORD AutomateThread(LPVOID pParam)
{
    try
    {
        g_Logger.WriteLog(_T("Running in console mode... Entering while(...)"));
        std::wstring port = Constants::MasterNodePort;
        std::wstring host = ServerHelper::GetHostName();
        std::wstring ip = ServerHelper::GetIP();
        std::wstring url = ServerHelper::BuildURL(ip, port);
        http::uri uri = http::uri(url);
        std::wstring address = uri.to_string();
        std::wstring name = _T("Master");

        std::vector<std::wstring> v = ServerHelper::GetIPs();
        for (std::wstring ipE : v)
        {
```

```
        g_Logger.WriteLog(ipE);
    }
    g_Logger.WriteLog(_T("Enum IP addresses ok"));

    //
    // Create the server instance
    //

    TCHAR sz[255];
    _stprintf_s(sz, _T("IP : %s"), ip.c_str());
    g_Logger.WriteLog(sz);

    g_Logger.WriteLog(_T("Creating server ip/port..."));
    TheServer client(address);
    g_Logger.WriteLog(_T("Creating server ip/port ok"));
    client._server = ip;
    client._port = port;
    client._name = name;
    client.Init();

    g_Logger.WriteLog(_T("Waiting..."));
    while (TRUE)
    {
        if (_Module.m_bStop)
        {
            std::wcout << _T("Running in console mode.") << std::endl;
            goto stop_service;
            break;
        }

        Sleep(5000);
        g_Logger.WriteLog(_T("MainThread Sleep..."));
    }

    // Main loop
```

Observations sur Docker

L'avantage de faire du conteneur avec du code C++ est que le code étant rapide. On met rarement le CPU en chauffe, la consommation de RAM est minimum, donc on ne paie presque rien. Les machines qui font tourner les containers Docker sont des monstres et l'exemple de mon code C++ fait que je suis à 0% de RAM et 0% de CPU.... Bref, pour faire tourner du bon code C++ de communication avec différentes versions de librairies, c'est une bonne option.

Conclusion

L'utilisation d'une application dans Docker n'est pas très compliquée à mettre en place. Il faut d'abord, créer un DockerFile avec une image d'OS et y copier ses binaires. C'est tout ! Ensuite, on ouvre les ports de communication et hop, le tour est joué. •